

AEMaaCS Deep-Dive Volume – Developers, Technical Architects, Admins

AEMaaCS Deep-Dive Volume

For Developers, Technical Architects, and Admins

This volume assumes you already know “basic AEM” and focuses on how to build, operate, and troubleshoot well-behaved applications on AEM as a Cloud Service (AEMaaCS).

1. Mental Model for AEMaaCS Engineering

1.1 Key differences vs. AEM 6.5/AMS

Cloud-native constraints that drive your design:

- **Clustered, always-on:** there are always multiple author/publish instances; rolling updates run old & new code side-by-side.
- **Immutable vs. mutable content:**
 - Immutable: /apps, /libs, Dispatcher configs – shipped via the all package.
 - Mutable: /content, /conf, /var, /home – persisted in a shared cloud datastore.
- **Ephemeral filesystem:**
 - Local disk is temporary; assume **pod can restart at any time**.
 - Use filesystem only for **small, request-scoped temp files**.

What this means for you:

- No “log into publish and tweak configs” – **everything is in Git** and deployed via Cloud Manager.
- No long-running state in memory; **everything important must be persisted** (JCR or external system).
- **Zero-downtime upgrades:** you must design for **forward- and backward-compatible content**.

Recommended references

- *AEM as a Cloud Service Development Guidelines* – especially “Code Must Be Cluster-Aware”, “State in Memory”, and “State on the Filesystem”.
[AEM as a Cloud Service Development Guidelines](#)

- *Architecture overview* for the logical and system architecture.

[An Introduction to the Architecture of Adobe Experience Manager as a Cloud Service](#)

2. Project & Repository Structure

2.1 Archetype-based project layout

Anchor your mental model on the AEM Project Archetype and then map real projects to it:

- **core** – Java:
 - Sling Models, servlets, OSGi services, schedulers, workflow steps.
- **ui.apps** – code deployed under /apps:
 - HTL, component definitions, clientlibs, dialogs, policies.
- **ui.content** – baseline content/config:
 - Only minimal **bootstrap content** (e.g., /conf templates/policies, sample pages if really needed).
- **ui.config** – OSGi configs & repo init:
 - Run-mode-aware configs, repo init for groups/ACLs, service users.
- **ui.frontend** – front-end build:
 - Webpack/Vite, TS/JS, SASS, compiled into clientlibs.
- **dispatcher** – Apache/Dispatcher for AEMaaCS:
 - conf.d, conf.dispatcher.d, vhosts, filters, cache rules.
- **it.tests / ui.tests** – integration + UI tests:
 - Executed in Cloud Manager pipelines.

Best practices

- **Keep ui.content thin:**
 - Don't ship large authoring trees; they cause conflicts with real content and complicate deployments.
- **Repo init over "bootstrap scripts":**
 - Co-locate your users/groups/ACLs and service users in ui.config repo-init, not in random Groovy scripts.
- **Dispatcher as first-class code:**
 - Treat dispatcher/ like Java code: code review, tests, and validation locally via the Dispatcher SDK.

References

- [AEM Project Structure](#)
 - [Getting Started Developing AEM Sites - WKND Tutorial](#)
-

3. Development Patterns & Anti-Patterns

3.1 Back-end (OSGi, Sling, Sling Models)

Patterns

- **Sling Models as your main backend API:**
 - Prefer `@Model(adaptables = Resource.class)` for page components and `@Model(adaptables = SlingHttpServletRequest.class)` for request-specific logic.
 - Use `@OSGiService` injection for services, `@ValueMapValue` for content.
- **Service APIs instead of "god" models:**
 - Create small, focused OSGi services; inject into Sling Models rather than doing everything in one model.
- **Cluster-safe schedulers & jobs:**
 - Use Sling Jobs / distributed tasks where possible.
 - If you must run a scheduler on one node, use Sling Discovery API to elect a leader.

Anti-patterns

- **Using `ResourceResolverFactory.getAdministrativeResourceResolver()`** anywhere (it's not available; use service users and `getServiceResourceResolver`).
- **Writing to /content in components:**
 - Author-side writes should be triggered via explicit actions (workflows, tools), not every render.
- **Blocking external calls in render path:**
 - Wrap external API calls behind caching or async processing; pre-populate data and store in JCR where feasible.

3.2 HTL and component structure

Patterns

- Keep HTL **presentation-only**:
 - No business logic, no heavy conditionals; use Sling Models for logic.
- Use **granular components**:

- Prefer a composable set of small, focused components instead of a few monoliths with dozens of checkboxes.
- Take advantage of **edit vs design dialog**:
 - Edit dialog = content-level options.
 - Design dialog/policy = structural/brand options (e.g., aspect ratios, allowed styles).

Anti-patterns

- Large data-sly-test chains with complex logic in HTL.
- Direct JCR access in HTL via `resourceResolver`. Use Sling Models.

References

- [Components Overview](#)
 - [Core Components Introduction](#)
-

4. Configuration Management & Environments

4.1 Run modes, OSGi config and secrets

Approach

- **Use `ui.config`** for OSGi configs:
 - Separate folders for `author`, `publish`, and `all` run modes.
- **Externalize secrets**:
 - Passwords, API keys, and client secrets belong in **environment variables or Adobe I/O/IMS credentials**, not in OSGi configs.
- Standardize **config naming**:
 - Use consistent PID patterns and comments; document which feature each config enables.

Patterns

- Create an `environment-configuration` page in your docs mapping:
 - Feature → OSGi PID → Environment variable / secret.

References

- [Implementing Applications for AEM as a Cloud Service](#)
 - [Generating Access Tokens for Server-Side APIs \(Legacy\)](#)
-

5. Integration Patterns

5.1 Outbound (AEM → external systems)

Patterns

- Wrap all external calls in an OSGi service with:
 - **Resilient HTTP client** (timeouts, retries, circuit breakers).
 - **Clearly modeled request/response DTOs**.
- **Decouple from request rendering**:
 - Pre-fetch data via workflows or scheduled jobs and materialize into JCR when possible.
- Use **technical accounts** and service credentials:
 - Avoid user-context tokens for server-to-server integrations.

5.2 Inbound (external → AEM)

Patterns

- Use **AEM APIs with access tokens** (OpenAPI endpoints, GraphQL, Sling selectors).
- Enforce **rate limits and authentication** at the edge (CDN, Dispatcher) plus AEM.

Reference

- [Implementing an AEM Connector](#)
-

6. Indexing & Query Design (Engineering View)

(Pair this with your indexing KT session.)

Best practices

- Design queries **from use-cases**, not from “what’s easy to write”.
- Always:
 - Use constrained paths (path=/content/...).
 - Prefer **property restrictions** on indexed properties.
 - Test queries with **Explain Query** and **Query Debugger** in SDK before shipping.

Common gotchas

- Queries that “work locally” but trigger **“Query without index detected”** alerts in Cloud due to:

- Wildcard property patterns (*foo*).
- Missing path constraints.
- Overly broad node-type queries.

Reference

- [Content Search and Indexing](#)
 - [Query and Indexing Best Practices](#)
-

7. Dispatcher & CDN (Engineering View)

Core rules

- **Least privilege:**
 - Block everything by default, explicitly allow only necessary paths and methods.
- **Layered caching:**
 - CDN for long-lived public content.
 - Dispatcher for HTML/JSON views, invalidated via flush agents / cache invalidation.
- Encode critical business rules in:
 - filters (security).
 - cache rules (what is cacheable).
 - headers (Cache-Control, Surrogate-Control).

Patterns

- Keep separate vhosts for:
 - Author, Publish, Preview.
 - Each public domain (per site) if required.
- Use the **Dispatcher SDK locally** and wire it into your dev loop.

Reference

- [Caching in AEM as a Cloud Service](#)
 - [Dispatcher Configurations in Adobe Experience Manager as a Cloud Service](#)
-

8. Testing & Cloud Manager

8.1 Testing pyramid

Layers

- Unit tests (Java/JS) – executed during Maven build.
- Integration tests (`it.tests`) – run against author/publish.
- UI tests (`ui.tests`, e.g., Cypress) – run in Cloud Manager pipeline.
- Performance & security tests – Cloud Manager quality gates.

Best practices

- Ensure every bug fix includes:
 - A **unit or integration test** that would have caught it.
 - If UI-related, a **UI test** in `ui.tests`.
- Treat **Cloud Manager failures** as first-class:
 - Fix code quality issues, don't just suppress rules.

References

- [CI/CD Pipelines](#)
 - [Cloud Manager and AEM as a Cloud Service](#)
-

9. Operations, Observability, and Troubleshooting

9.1 Logs and Developer Console

Workflow

1. Reproduce issue in lowest environment (local SDK → dev → stage).
2. Use **Developer Console** for:
 - Recent logs (error, request, custom).
 - Health checks and status dumps.
3. Correlate logs across:
 - AEM error logs.
 - Dispatcher access/error logs.
 - CDN logs if available.

Reference

- [Operations and Maintenance on Experience Manager as a Cloud Service](#)
- [Debugging AEM as a Cloud Service with the Developer Console](#)

9.2 Performance & query troubleshooting

Tools

- Query Performance/Explain Query in SDK.
- Cloud Manager performance test results.
- Application profiling (locally) with JFR or similar.

Patterns

- Start with **queries and indexing**, not CPU tuning.
 - Use feature flags or configs for **expensive features** to disable quickly in non-prod when testing.
-

10. Security, IAM, and Permissions (Technical View)

Principles

- Single source of truth for identities = **Adobe Admin Console**; AEM groups are *permission buckets*.
- Avoid per-user ACLs; use **role-based groups**.
- Use **service users** for all automated content changes.

References

- [AEM as a Cloud Service - Terminology](#)
 - [Security Overview for AEM as a Cloud Service](#)
-

11. Real-World Scenario Patterns

11.1 “We have to add a new integration quickly”

Checklist:

1. Agree on **data ownership** and system of record.
2. Define a clear **failure strategy** (fail-open vs fail-closed).
3. Implement **feature flag** via OSGi config.
4. Add **structured logging** around the integration.
5. Validate in **RDE**, then non-prod pipeline, then prod.

11.2 "Random 5xx errors after a deployment"

Checklist:

1. Check Cloud Manager deployment logs for errors/warnings.
 2. Check Developer Console logs for:
 - Missing OSGi configs.
 - Indexing errors.
 - Out-of-memory or thread pool exhaustion.
 3. Roll back via **previous image** (pipeline) rather than hotfixing in-place.
 4. Add test coverage for the root cause.
-

References (Engineering & Admin Volume)

- [AEM as a Cloud Service Development Guidelines](#)
- [Implementing Applications for AEM as a Cloud Service](#)
- [An Introduction to the Architecture of Adobe Experience Manager as a Cloud Service](#)
- [AEM development playlists](#)
- [Operations and Maintenance on Experience Manager as a Cloud Service](#)
- [Introduction to Adobe Experience Manager as a Cloud Service - Terminology](#)
- [Cloud Manager and AEM as a Cloud Service](#)